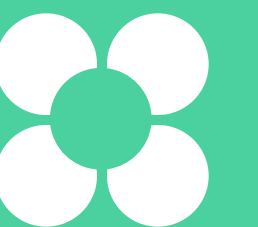
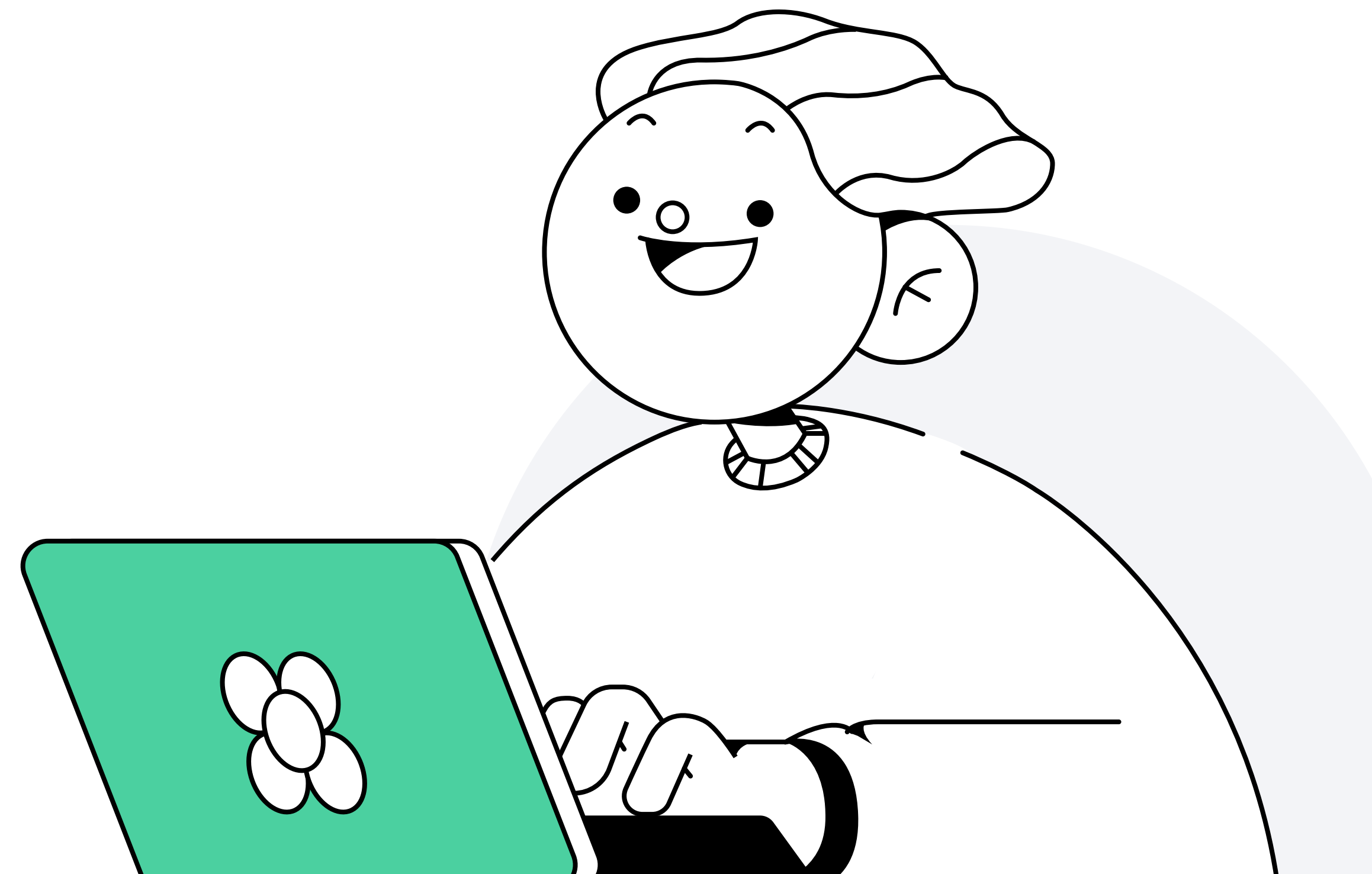


Исключения, `stacktrace`



План занятия

- 1 Обработка ошибок в Java
- 2 Иерархия классов ошибок
- 3 Создание собственных ошибок



Обработка ошибок Java



Ошибки в реальном мире

Теоретически водитель автомобиля может по ошибке залить в него не тот вид топлива (дизель вместо бензина) или топливо с более низким октановым числом. Или он может перекачать колёса и тем самым вывести автомобиль из строя.

Программы с точки зрения сложности можно соотнести с автомобилями, но часто они гораздо гибче в разработке, и в них можно определять поведение при появлении ошибки



Ошибки в программах

Создадим программу,
которая выводит на экран
десятый символ в строке.

Но в строку поместим
фразу короче 10 символов.

Давайте посмотрим, что
произойдёт в этом случае

```
public class Main {  
  
    public static void main(String[] args) {  
        System.out.println("Hello");  
  
        String s = "Hello";  
        char c = get10th(s);  
        System.out.println("Ответ был: " + c);  
  
        System.out.println("Bye");  
    }  
  
    public static char get10th(String s) {  
        System.out.println("Начинаем извлечение символа");  
        char c = s.charAt(10);  
        System.out.println("Вынули символ");  
        return c;  
    }  
}
```

Ошибки в программе

Java выдаст нам такую ошибку:

```
Exception in thread "main" java.lang.StringIndexOutOfBoundsException Create  
Breakpoint : String index out of range: 10  
  at java.base/java.lang.StringLatin1.charAt(StringLatin1.java:48)  
  at java.base/java.lang.String.charAt(String.java:711)  
  at Main.get10th(Main.java:15)  
  at Main.main(Main.java:7)
```

где:

- `StringIndexOutOfBoundsException` — класс объекта-отчёта, который Java сформировал, когда мы попытались выполнить недопустимую операцию
- `String index out of range: 10` — сообщение об ошибке
- `at java.base/java.lang.StringLatin1.charAt(StringLatin1.java:48)` и строки ниже — **stacktrace** — специальная стопка вызовов, с помощью которой мы можем понять, в каком месте программы произошла ошибка

Ошибки в программе

Сталкиваясь с ошибкой, Java, по сути, убивает нашу программу. Этот процесс происходит не одномоментно: сначала Java поймёт, что не может завершить какой-то метод, сформирует отчёт об ошибке, далее будет пытаться выйти из всех вызовов, в которых находится ошибка. Так Java убьёт все связанные методы и выведет ошибку на экран.

Задача программиста — уметь предвидеть появление ошибок и правильно их обрабатывать. Например, в одних случаях необходимо завершать работу программы, а в других — продолжать её выполнение и запрашивать ввод корректных данных

Ошибки в программе

В языке Java ошибки в программах называются исключительными ситуациями или исключениями (на английском **Exception**).



Появление исключения или создание исключительной ситуации при работе программы в Java принято называть выбросом исключения (на английском **throws Exception**).



А способы обработки исключительной ситуации называют поимкой или перехватом исключения (на английском **catch Exception**).

Обработка исключительной ситуации

Добавим в нашу программу обработку исключения и выведем пользователю сообщение об ошибке ввода.

Для этого используем специальную синтаксическую конструкцию **try - catch**.

- try начинает блок кода, который потенциально может привести к ошибке
- catch начинает блок кода, перехватывающий ошибки
- finally используется, если программе требуется выполнить какой-то код вне зависимости от того, появилось исключение или нет

```
public class Main {  
  
    public static void main(String[] args) {  
        System.out.println("Hello");  
  
        try {  
            String s = "Hello";  
            char c = get10th(s);  
            System.out.println("Ответ был: " + c);  
        } catch (Exception e) {  
            System.out.println("ОШИБКА");  
        } finally {  
  
        }  
  
        System.out.println("Bye");  
    }  
  
    public static char get10th(String s) {  
        System.out.println("Начинаем извлечение символа");  
        char c = s.charAt(10);  
        System.out.println("Вынули символ");  
        return c;  
    }  
}
```


Обработка исключительной ситуации

В случае ошибки результатом работы программы будет наше сообщение **ОШИБКА**.

Мы обработали только один класс исключений — **Exception**.

Однако существует целая иерархия исключений,
и все они наследуются от базового класса **Throwable**

Способы обработки исключений

Есть несколько способов обрабатывать исключительные ситуации

- ① Не обрабатывать (только unchecked)
- ② Использовать операторы **try/catch/finally**
- ③ Объявить в сигнатуре метода* **throws**
- ④ Выбросить ошибку в коде: **throw new Exception**

*Сигнатура метода — это имя метода и список его аргументов

1. Не обрабатывать (только unchecked)

Рассмотрим пример. Ниже программа производит деление на ноль:

```
class Main {  
    public static void main(String[] args) {  
        int value = 100;  
        System.out.println(value/0);  
    }  
}
```

При делении любого числа на ноль возникнет ошибка `ArithmeticException` — этот класс исключения наследуется от `RuntimeException` и является `unchecked` (компилятор не будет проверять во время процесса компиляции, обрабатывается это исключение или нет).

Во время работы мы получим ошибку:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at Main.main(Main.java:4)
```


2. Использовать операторы `try/catch/finally`

Этим способом мы уже пользовались ранее. В Java есть три ключевых слова для обработки исключений:

- **try** используется для отметки начала блока кода, который потенциально может привести к исключению и ошибке
- **catch** используется для отметки начала блока кода, предназначенного для перехвата и обработки исключений
- **finally** используется для отметки начала блока кода, который является дополнительным

Можно использовать несколько блоков `catch` и располагать их друг за другом.

Блок `finally` помещается после последнего блока `catch`. Он необязательный, но используется, если в программе требуется выполнить какой-то код вне зависимости от того, появилось исключение или нет. Обычно в этом блоке происходит закрытие и освобождение ресурсов: файла, сетевого соединения и т. д.

2. Использовать операторы try/catch/finally

Рассмотрим программу ввода двух чисел и будем перехватывать исключение деления на ноль:

```
class Main {  
    public static void main(String[] args) {  
        try {  
            Scanner scanner = new Scanner(System.in);  
            System.out.println("Введите первое число:");  
            int value1 = scanner.nextInt();  
            System.out.println("Введите второе число:");  
            int value2 = scanner.nextInt();  
            System.out.println(value1 / value2);  
        } catch (ArithmeticException exception) {  
            System.out.println("Деление на ноль");  
        }  
    }  
}
```

2. Использовать операторы try/catch/finally

Мы обернули «опасный код» операторами **try/catch**.

В блок **try** поместили весь код, который может вызвать ошибку, а в блок **catch** — действие, которое хотим выполнить при возникновении ошибки.

В нашем примере используется класс **Scanner**, который может работать не только с консолью, но также с файлами и другими ресурсами, требующими принудительного закрытия. Поэтому независимо от того, возникнет ошибка или нет, нужно всегда выполнять блок кода **finally**:

```
Scanner scanner = new Scanner(System.in);  
try {  
    //code  
} catch (ArithmeticException exception) {  
    System.out.println("Деление на ноль");  
} finally {  
    scanner.close();  
}
```


2. Использовать операторы try/catch/finally

Если мы используем в блоке try/catch какой-либо внешний ресурс (читаем файл, используем scanner), важно после использования закрыть канал доступа к этому ресурсу и освободить его, например, вызвав метод close().

Для автоматического закрытия ресурсов хорошей практикой считается реализация интерфейса closable в классе, работающем с ресурсом. Для классов, реализующих этот интерфейс, есть возможность использовать специальный синтаксис try/catch with resources)

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.println("Введите первое число:");
            String value1 = scanner.nextLine();
            System.out.println("Введите второе число:");
            String value2 = scanner.nextLine();

            int parseValue1 = Integer.parseInt(value1);
            int parseValue2 = Integer.parseInt(value2);
            System.out.println(parseValue1 / parseValue2);
        } catch (RuntimeException exception) {
            System.out.println("Введённые данные некорректны");
        }
    }
}
```


2. Использовать операторы try/catch/finally

Можно располагать несколько операторов catch друг после друга.

Изменим предыдущий пример и будем считывать из консоли строковые значения. При этом у нас может возникнуть ошибка преобразования строки в число — **NumberFormatException**

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.println("Введите первое число:");
            String value1 = scanner.nextLine();
            System.out.println("Введите второе число:");
            String value2 = scanner.nextLine();

            int parseValue1 = Integer.parseInt(value1);
            int parseValue2 = Integer.parseInt(value2);
            System.out.println(parseValue1 / parseValue2);
        } catch (ArithmeticException exception) {
            System.out.println("Деление на ноль");
        } catch (NumberFormatException exception) {
            System.out.println("Ошибка преобразования значения");
        } finally {
            scanner.close();
        }
    }
}
```

2. Использовать операторы try/catch/finally

Если мы знаем, что действия в блоке **catch** будут одинаковы при разных исключениях, то можем их **объединить в один блок с помощью оператора «|»**

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.println("Введите первое число:");
            String value1 = scanner.nextLine();
            System.out.println("Введите второе число:");
            String value2 = scanner.nextLine();

            int parseValue1 = Integer.parseInt(value1);
            int parseValue2 = Integer.parseInt(value2);
            System.out.println(parseValue1 / parseValue2);
        } catch (ArithmeticException | NumberFormatException
exception) {
            System.out.println("Введённые данные некорректны");
        } finally {
            scanner.close();
        }
    }
}
```

2. Использовать операторы try/catch/finally

Если мы хотим ловить все исключения, наследуемые от одного общего класса (как было в примере выше – **RuntimeException**), можем указать только его в качестве аргумента для перехвата исключения

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.println("Введите первое число:");
            String value1 = scanner.nextLine();
            System.out.println("Введите второе число:");
            String value2 = scanner.nextLine();

            int parseValue1 = Integer.parseInt(value1);
            int parseValue2 = Integer.parseInt(value2);
            System.out.println(parseValue1 / parseValue2);
        } catch (RuntimeException exception) {
            System.out.println("Введённые данные некорректны");
        } finally {
            scanner.close();
        }
    }
}
```


2. Использовать операторы try/catch/finally

Если при описании исключений указать класс-родитель раньше, то блок с исключением, следующий после него, не будет вызван.

В примере ниже блок с ArithmeticException не будет вызван:

```
try {  
    //code  
} catch (RuntimeException exception) {  
    System.out.println("Введённые данные некорректны");  
} catch (ArithmeticException exception) {  
    System.out.println("Деление на ноль");  
}
```

2. Использовать операторы try/catch/finally

Но обратное расположение допустимо и часто используется. Сначала в коде описывается класс-наследник, а за ним более общий (класс-родитель):

```
try {  
    //code  
} catch (ArithmeticException exception) {  
    System.out.println("Деление на ноль");  
} catch (RuntimeException exception) {  
    System.out.println("Введённые данные некорректны");  
}
```


3. Объявить в сигнатуре метода `throws`

Если какой-либо метод в нашей программе может выбросить исключение **checked**, то компилятор потребует от нас обязательно его обработать (как мы делали это раньше с помощью блоков **try/catch**) или объявить возможность возникновения исключения при описании метода.

После завершения описания аргументов нужно добавить оператор **throws**:

```
public void myMethod(int value1, int value2) throws ParseException {}
```

3. Объявить в сигнатуре метода throws

Через запятую можно перечислить исключения, которые могут возникнуть в методе:

```
public void myMethod(int value1, int value2) throws ParseException, IOException {}
```

Но можно объединить родительским исключением:

```
public void myMethod(int value1, int value2) throws Exception {}
```

Исключения, объявленные с использованием оператора throws, будут выброшены в поток ошибок (по умолчанию — консоль) и выведены на экран

3. Объявить в сигнатуре метода throws

Пример объявления **checked**-исключения для метода преобразования даты, введённой пользователем (метод **parseDate**):

```
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        try {
            Scanner scanner = new Scanner(System.in);
            System.out.println("Введите дату в формате 'dd MM yyyy'");
            String inputDate = scanner.nextLine();
            Date date = parseDate(inputDate);
            System.out.println(date);
        } catch (ParseException exception) {
            System.out.println("Ошибка разбора даты");
        }
    }

    private static Date parseDate(String inputDate) throws ParseException {
        SimpleDateFormat dateFormat = new SimpleDateFormat("MM dd yyyy");
        return dateFormat.parse(inputDate);
    }
}
```


4. Выбросить ошибку в коде: `throw new Exception`

Если нам требуется прекратить работу программы (например, пользователь ввёл некорректные данные), мы можем сами инициировать создание и выброс исключения.

Для выброса исключения нужно в методе объявить оператор `throw` и создать исключение, например: `throw new Exception («Сообщение»);`

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Введите первое число:");
        int value1 = scanner.nextInt();
        System.out.println("Введите второе число:");
        int value2 = scanner.nextInt();

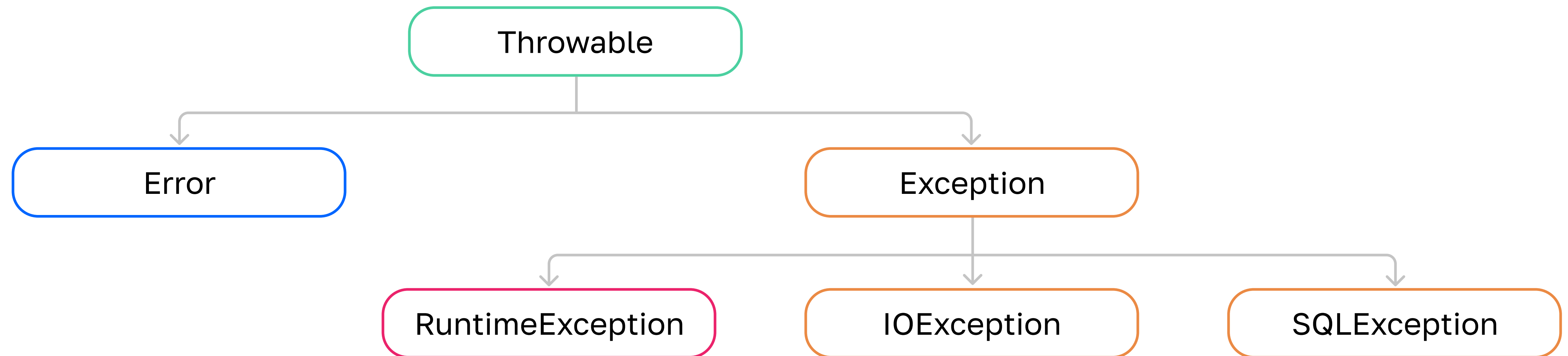
        if (value1 == 0 || value2 == 0) {
            throw new Exception("Значения не должны быть меньше нуля");
        }
        System.out.println(value1 / value2);
    }
}
```

Иерархия классов ошибок



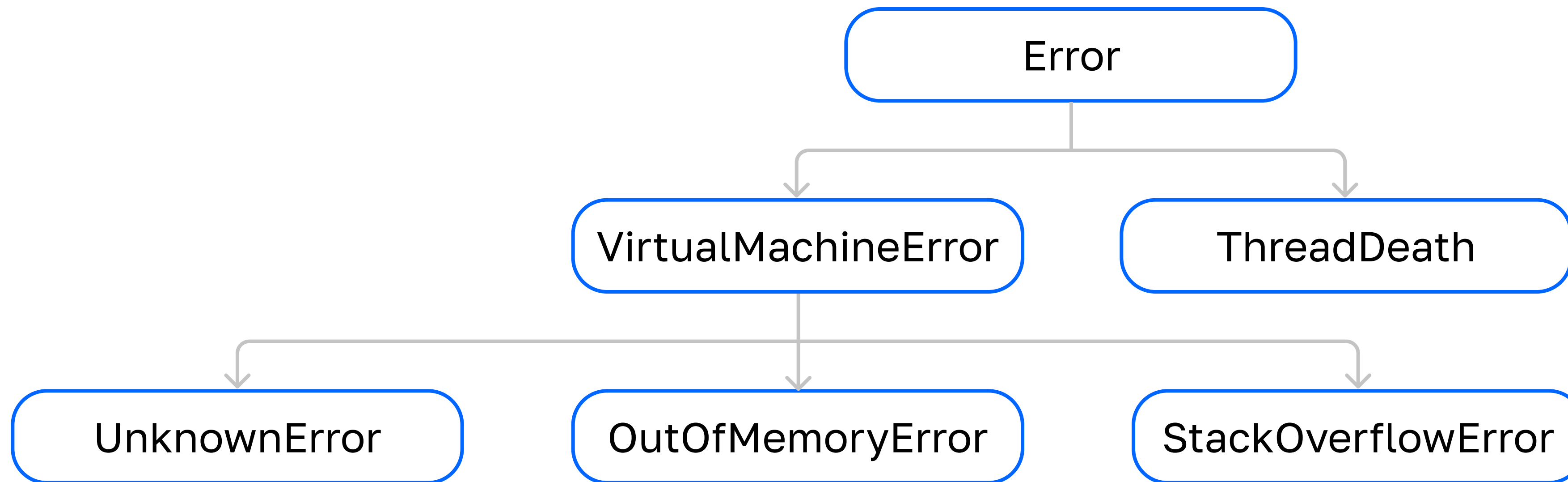
Иерархия исключений

Базовый класс всех исключений в Java — это класс Throwable, фактически от него наследуются все ошибочные ситуации



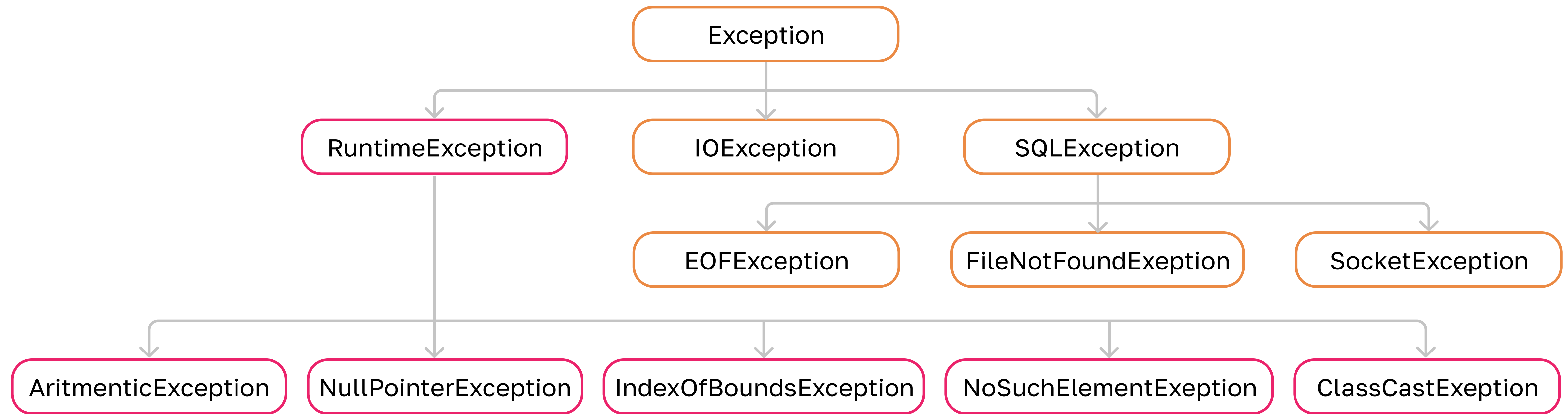
Иерархия исключений – Error

Ошибки работы виртуальной машины Java и её окружения

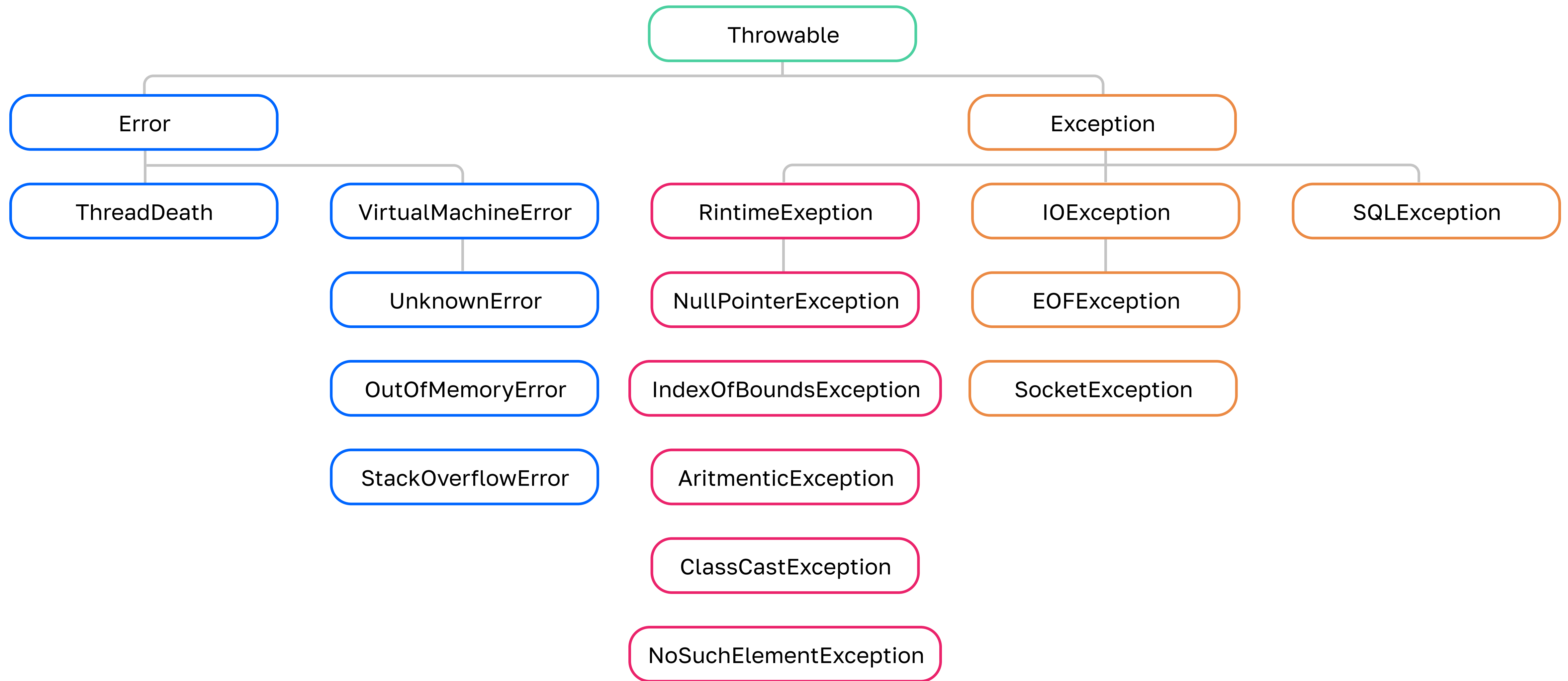


Иерархия исключений – Exception

Ошибки, связанные с работой приложения



Иерархия исключений



Иерархия исключений

- **Throwable** — базовый класс, от которого наследуются все остальные исключительные ситуации и ошибки (**checked**)
- **Error** — ошибки, связанные с работой JVM и её окружением; например, **VirtualMachineError**, **OutOfMemoryError** (**unchecked**). Такие ошибки лучше не ловить
- **Exception** — (принудительно) проверяемые исключения. Например, **IOException**, **ClassNotFoundException** (**checked**)
- **RuntimeException** — непроверяемые исключения работы приложения. Например, **ArithmeticException**, **ClassCastException** (**unchecked**)

Checked- и unchecked-исключения

Исключения делятся на два типа

- ① **Checked** (проверяемые) — исключения, которые обязательно нужно обработать или указать в описании метода, что в нём может возникнуть такого типа ошибка. Если этого не сделать, то компилятор не скомпилирует код и вернёт ошибку компиляции

Это всё исключения, наследуемые от Throwable и Exception.

- ② **Unchecked** (непроверяемые) — исключения, которые не обязательно обрабатывать, но они могут возникнуть во время выполнения программы

Это всё исключения, наследуемые от Error и RuntimeException

Создание собственных ошибок

Создание собственных ошибок

Чтобы создать своё исключение, нужно унаследовать свой класс от класса-исключения.

Для создания своего **checked exception** можно отнаследоваться от классов **Throwable** или **Exception**, а для создания **unchecked exception** — от класса **RuntimeException**.

Исключения создаются, чтобы пользователь, который неправильно вызывает наш метод, получал говорящее сообщение об ошибке

Создание своего исключения

Напишем метод, принимающий текст, вынимающий из него имя и фамилию и проверяющий, совпадают ли они по количеству букв.

Заведомо передадим в программу некорректный текст — оставим только фамилию:

```
public class Main {  
  
    public static void main(String[] args) {  
        System.out.println(sameLength("Petrov"));  
    }  
  
    public static boolean sameLength(String text) { // "Petrov Petya"  
        String[] parts = text.split(" "); // ["Petrov", "Petya"]  
        String name = parts[1];  
        String lastName = parts[0];  
        return name.length() == lastName.length();  
    }  
} // в результате метод выдает непонятную ошибку
```

Создание своего исключения

Создадим свой тип исключений `NameInputMismatchException` на основе класса **`RuntimeException`** со своим сообщением.

Этот тип исключений будет использоваться всегда, когда команде передаётся неправильный текст, не состоящий из имени и фамилии:

```
public class NameInputMismatchException extends RuntimeException {  
  
    public NameInputMismatchException(String text) {  
        super("В параметре надо указать имя и фамилию, но было указано: " + text);  
    }  
}
```

Создание своего исключения

Добавим наш класс исключений при написании программы:

```
public class Main {  
  
    public static void main(String[] args) {  
        System.out.println(sameLength("Petrov"));  
    }  
  
    public static boolean sameLength(String text) { // "Petrov Petya"  
        String[] parts = text.split(" "); // ["Petrov", "Petya"]  
        if (parts.length != 2) {  
            throw new NameInputMismatchException(text);  
        }  
        String name = parts[1];  
        String lastName = parts[0];  
        return name.length() == lastName.length();  
    }  
}
```


Создание своего исключения

Проверим, как теперь будет работать наша программа при неправильном вводе имени и фамилии:

```
Exception in thread "main" NameInputMismatchException: В параметре надо указать  
имя и фамилию, но было указано: Petrov  
    at Main.sameLength(Main.java:10)  
    at Main.main(Main.java:4)
```

Как происходит выброс ошибок (исключений)

Ранее мы уже упоминали понятие **stacktrace**. Остановимся на нём подробнее.

Во время работы JVM в каждый момент времени хранит всю цепочку вызовов методов, складывая их в специальную коллекцию `stack`. Каждый новый метод, вызываемый JVM, попадает в `stack` до момента своего завершения и возвращения результата. В момент возникновения исключительной ситуации JVM возвращает значение этой коллекций `stack`, в котором можно увидеть класс и метод возникновения ошибки. Эта коллекция называется **stacktrace**:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at Main.main(Main.java:4)
```

Из этого `stacktrace` видно, что ошибка произошла в методе `main` класса `Main` в строке 4

Пример stacktrace

В программе может возникнуть ситуация, когда ошибка в одном из методов создаст исключение в другом. В этом случае stacktrace будет выглядеть похожим образом.

```
javax.servlet.ServletException: Send message error
    at com.example.myproject.OpenSessionInViewFilter.doFilter(OpenSessionInViewFilter.java:60)
    at org.mortbay.jetty.servlet.ServletHandler$CachedChain.doFilter(ServletHandler.java:1157)
    at com.example.myproject.ExceptionHandlerFilter.doFilter(ExceptionHandlerFilter.java:28)
    at org.mortbay.thread.QueuedThreadPool$PoolThread.run(QueuedThreadPool.java:582)
Caused by: com.example.myproject.MyProjectServletException
    at com.example.myproject.MyServlet.doPost(MyServlet.java:169)
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:727)
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:820)
Caused by: org.hibernate.exception.ConstraintViolationException: could not insert: [com.example.myproject.MyEntity]
    at org.hibernate.exception.SQLStateConverter.convert(SQLStateConverter.java:96)
    at com.example.myproject.MyServlet.doPost(MyServlet.java:164)
    ... 32 more
Caused by: java.sql.SQLException: Violation of unique constraint MY_ENTITY_UK_1: duplicate value(s) for column(s) MY_COLUMN
in statement [...]
    at org.hsqldb.jdbc.Util.throwError(Unknown Source)
    ... 54 more
```

Ошибка ServletException возникла из-за MyProjectServletException, которая произошла из-за ConstraintViolationException, а самой первой ошибкой, из-за которой прекратилась работа программы, была SQLException. В сообщении ошибки выделены Caused by

Stacktrace

Stacktrace можно вывести у любого объекта Throwable и всех его потомков, например Exceptions, вызвав метод **printStackTrace()** или **getStackTrace()**. Такое бывает нужно, если мы хотим гарантировать вывод stacktrace, например, в terminal или console.

Вывод **stacktrace** позволяет быстро понять, в каком методе произошла ошибка, без дополнительного вывода переменных, точек остановки программы и сообщений логирования.

При возникновении ошибки стоит первым делом посмотреть, что было выведено в **stacktrace**

**Спасибо
за внимание!**

